Introduction to Building Large Language Models

A Comprehensive Study Guide

Lecture by Yann Dubois
Stanford CS229: Machine Learning
August 13th, 2024

Compiled: January 28, 2026

Abstract

This document provides comprehensive study notes on building Large Language Models (LLMs), covering the complete pipeline from pretraining to post-training. The notes synthesize lecture content from Stanford's CS229 course, covering five critical components: architecture, training algorithms and losses, data processing, evaluation methodologies, and system implementation. While architecture receives significant academic attention, this guide emphasizes the practical aspects—data, evaluation, and systems—that are crucial for successful LLM deployment in real-world applications.

Contents

1 Introduction to Large Language Models

1.1 What are LLMs?

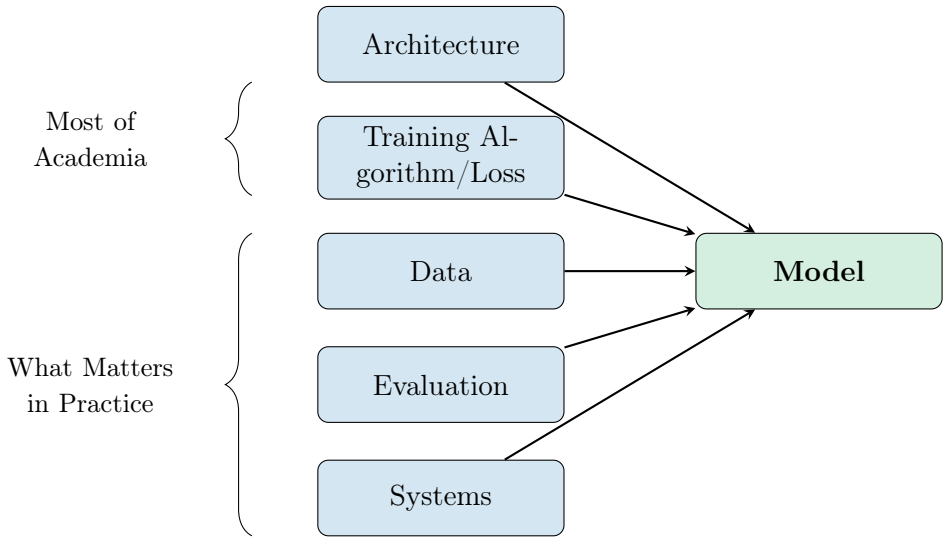
Large Language Models

Large Language Models (LLMs) are neural network-based systems trained on vast amounts of text data to understand and generate human-like language. Notable examples include:

- **ChatGPT** (OpenAI)
- **Claude** (Anthropic)
- **Gemini** (Google)
- **Llama** (Meta)

1.2 Five Critical Components of LLM Training

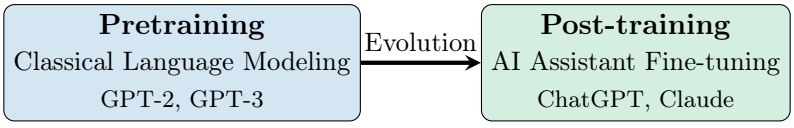
The development of effective LLMs requires careful attention to five interconnected components:



Focus of These Notes

While academia predominantly focuses on **architecture** and **training algorithms**, industry practitioners recognize that **data**, **evaluation**, and **systems** are equally—if not more—critical for practical success. This guide emphasizes these often under-documented aspects.

1.3 Two Training Paradigms



2 Pretraining: Foundation of LLMs

2.1 Language Modeling Fundamentals

Language Model

A **language model** is a probability distribution over sequences of tokens (or words):

$$p(x_1, x_2, \ldots, x_L)$$

where $x_1, \ldots, x_L$ represent tokens in a sequence of length L.

2.1.1 Examples of Language Model Probabilities

Consider the following sentences and their associated probabilities:

1. `"The mouse ate the cheese"` $p = 0.02$
   - Grammatically correct, semantically reasonable
2. `"The the mouse ate cheese"` $p = 0.0001$
   - Contains grammatical error (repeated "the")
   - Requires **syntactic knowledge**
3. `"The cheese ate the mouse"` $p = 0.001$
   - Grammatically correct but semantically unusual
   - Requires **semantic knowledge**

Generative Property

Language models are **generative models**: once we have a probability distribution $p(x_1, \ldots, x_L)$, we can sample from it to generate new sentences.

2.2 Autoregressive Language Models

Autoregressive Language Model

An **autoregressive (AR) language model** decomposes the joint probability distribution using the chain rule of probability:

$$p(x_1, \ldots, x_L) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdots p(x_L|x_1, \ldots, x_{L-1})$$

More concisely:

$$p(x_1, \ldots, x_L) = \prod_{i=1}^{L} p(x_i|x_{1:i-1})$$

where $x_{1:i-1}$ denotes the context (all previous tokens).

Key Insight

This decomposition involves **no approximation**—it is simply an application of the chain rule of probability. The key insight is:

We only need a model that can predict the next token given past context!

2.2.1 Advantages and Limitations

Advantages:

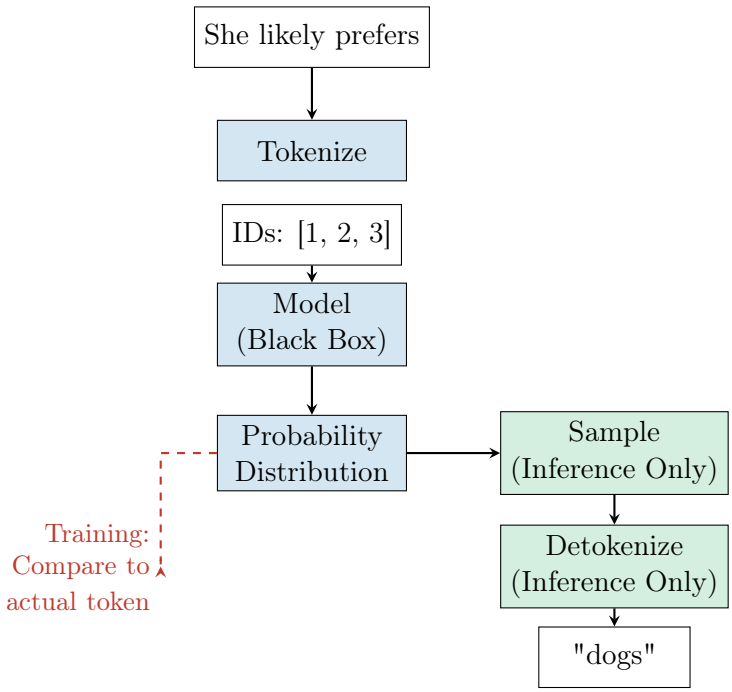
- Simple and effective
- Well-studied theoretical properties
- Natural fit for sequential data

Limitations:

- Sequential generation (for loop over tokens)
- Generation time scales linearly with sequence length
- Cannot generate tokens in parallel

2.3 The Prediction Task

The core task of autoregressive language modeling is **predicting the next word**. Here's the process:



Training vs. Inference

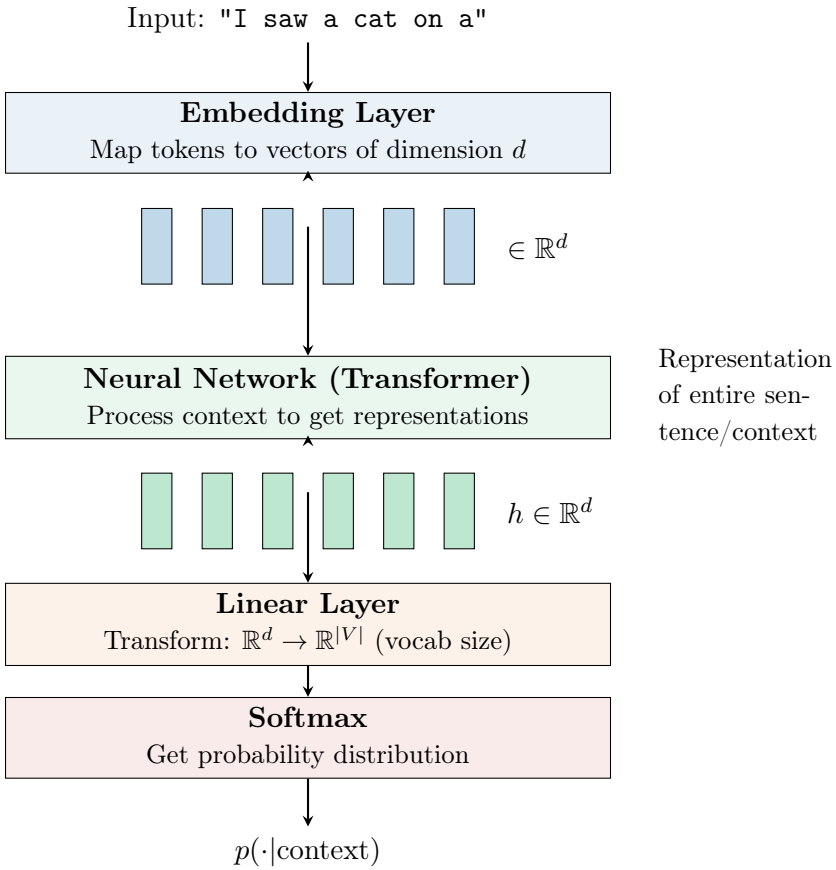
- **Training:** Only requires predicting the most likely token and comparing to the actual token
- **Inference:** Requires sampling from the distribution and detokenizing

3 Neural Language Model Architecture

Transformer Architecture

All modern LLMs are based on **transformers** or variants thereof. While we won't detail the architecture here (as extensive resources exist online), understanding the high-level flow is crucial.

3.1 Architecture Pipeline



Vocabulary Size

The output dimension must equal the vocabulary size $|V|$. This is crucial because:

- Each output dimension corresponds to one possible token
- Adding new tokens requires modifying the output layer
- Vocabulary size is fixed during training (typically 30,000–100,000+ tokens)

4 Training Loss and Optimization

4.1 Cross-Entropy Loss

The training of autoregressive language models is fundamentally a **token classification task**.

Cross-Entropy Loss

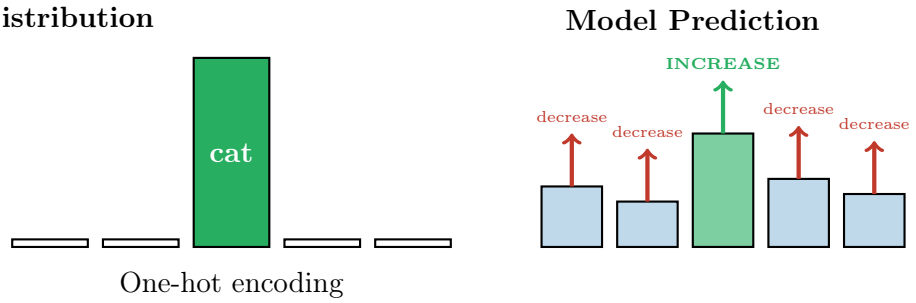
For a training example with target token (e.g., "cat"), the loss is:

$$\mathcal{L} = -\log p(\text{cat}|\text{context})$$

More generally, for a sequence $x_{1:L}$:

$$\mathcal{L}(x_{1:L}) = -\sum_{i=1}^{L} \log p(x_i|x_{1:i-1})$$

4.1.1 Visual Representation of Loss



4.2 Equivalence to Maximum Likelihood

Loss-Likelihood Equivalence

Minimizing the cross-entropy loss is **exactly equivalent** to maximizing the log-likelihood of the text:

$$\begin{aligned}\max_{\theta} \prod_{i=1}^{L} p(x_i|x_{1:i-1};\theta) &= \max_{\theta} \sum_{i=1}^{L} \log p(x_i|x_{1:i-1};\theta) \\ &= \min_{\theta} \left[-\sum_{i=1}^{L} \log p(x_i|x_{1:i-1};\theta) \right] \\ &= \min_{\theta} \mathcal{L}(x_{1:L};\theta)\end{aligned}$$

where θ represents the model parameters.

5 Tokenization: Converting Text to Tokens

Tokenizer

A **tokenizer** is a function that maps text to a sequence of token indices:

$$\text{tokenizer} : \text{Text} \rightarrow \text{Token IDs}$$

5.1 Why Tokenizers?

Motivation for Tokenization

Tokenizers solve two critical problems:

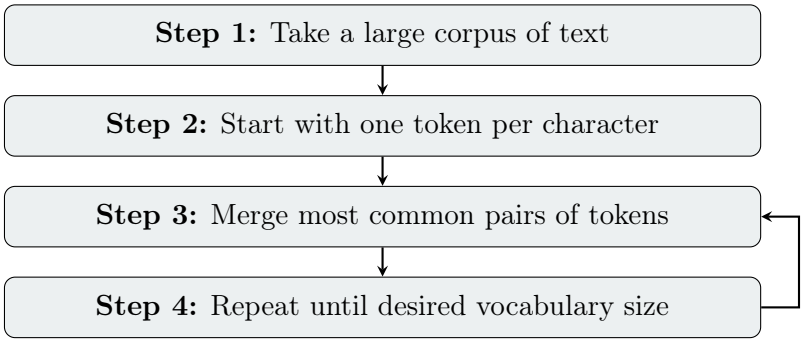
1. **More general than words**
   - Handles typos and misspellings
   - Works across languages without word boundaries (e.g., Thai)
   - Deals with compound words and rare words
2. **Shorter sequences than characters**
   - Character-level would result in very long sequences
   - Transformer complexity: $O(L^2)$ where L is sequence length
   - Tokens represent ∼3–4 letters on average

5.2 Byte Pair Encoding (BPE)

Byte Pair Encoding

BPE is one of the most common tokenization algorithms. It learns tokens by iteratively merging the most frequent pairs of tokens in a corpus.

5.2.1 BPE Training Algorithm



5.2.2 BPE Example

Consider training BPE on a small corpus:

Initial Corpus:

```
tokenizer tokenizer tokenizer text text
```

Iteration 1: Character-level tokenization

t o k e n i z e r
t o k e n i z e r

Iteration 2: Merge "t" + "o" (appears 3× as "to")

to k e n i z e r

Iteration 3: Merge "to" + "k" (appears 3× as "tok")

tok e n i z e r

Continue merging...

- "tok" + "e" → "toke" (2 times)
- "toke" + "n" → "token" (2 times)
- Final vocabulary includes: `{t, o, k, e, n, ..., to, tok, toke, token, ...}`

5.3 Real-World Tokenization Example

GPT-3 Tokenization Example

Input: `"tokenizer"`

Tokenized: `token` `izer`

The word "tokenizer" is split into two tokens: "token" and "izer", showing how common subwords become their own tokens.

5.4 Important Considerations

Tokenization Challenges

1. **Pre-tokenization:** Spaces and punctuation are typically handled separately for efficiency (English-specific optimization)
2. **Token preference:** When encoding text, always use the *largest* matching token from the vocabulary
3. **Vocabulary is fixed:** Adding new tokens requires retraining the output layer
4. **Numbers are problematic:** A number like "327" might have its own token, preventing compositional understanding
5. **Code tokenization:** Special handling needed for indentation (e.g., 4 spaces in Python)

Future Direction

Many researchers believe we should move toward **character-level or byte-level** tokenization once we have architectures that scale better than $O(L^2)$ with sequence length.

6 Evaluation Methodologies

6.1 Perplexity

Perplexity

Perplexity (PPL) is an evaluation metric derived from the validation loss:

$$\text{PPL}(x_{1:L}) = 2^{\frac{1}{L}\mathcal{L}(x_{1:L})} = \left[\prod_{i=1}^{L} p(x_i|x_{1:i-1})\right]^{-1/L}$$

6.1.1 Properties and Intuition

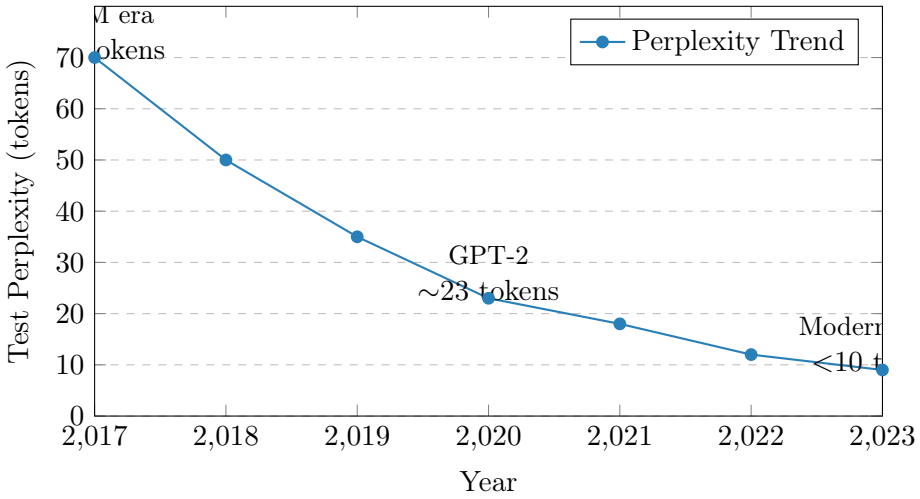
Perplexity Interpretation

- **Range:** $1 \leq \text{PPL} \leq |V|$ (vocabulary size)
- **Lower bound (PPL = 1):** Perfect prediction—no hesitation
- **Upper bound (PPL = $|V|$):** Uniform distribution—maximum uncertainty
- **Intuition:** Number of tokens the model is "hesitating between"

6.1.2 Why Use Perplexity Instead of Loss?

1. **Interpretability:** Humans understand "hesitating between 10 words" better than "loss = 2.3"
2. **Length-independent:** Average per-token metric
3. **Base-independent:** Exponentiation removes dependence on logarithm base
4. **Vocabulary-scale units:** Results in interpretable units

6.1.3 Historical Progress



Progress Summary

Between 2017 and 2023, perplexity decreased from ∼70 tokens to <10 tokens—a dramatic improvement meaning models went from hesitating between 70 possible words to fewer than 10.

Current Status of Perplexity

Perplexity is **no longer used for academic benchmarking** because:

- Depends on the tokenizer used
- Depends on the evaluation dataset
- Not comparable across different models with different tokenizers

However, it remains **crucial for development** within organizations.

6.2 Aggregate Benchmarks

Modern evaluation aggregates many **automatically evaluatable benchmarks**.

6.2.1 Popular Benchmark Suites

HELM
(Holistic Evaluation of Language Models)
Stanford

Open LLM Leaderboard
HuggingFace

Both aggregate scores across multiple tasks:
Question answering, reasoning, knowledge, etc.

6.3 MMLU: A Representative Benchmark

MMLU

Massive Multitask Language Understanding (MMLU) is one of the most trusted pretraining benchmarks. It consists of multiple-choice questions across 57 subjects.

6.3.1 Example Question

MMLU Example: Astronomy

Question: What is true for Type-1a supernovae?

- ☒ **A.** They always have the same peak luminosity (CORRECT)
- ☐ B. They occur in binary star systems only
- ☐ C. They are caused by core collapse
- ☐ D. They produce gamma-ray bursts

6.3.2 Evaluation Methodologies for MMLU

1. **Method 1: Likelihood Comparison**
   - Compute $p(\text{A}|\text{question})$, $p(\text{B}|\text{question})$, etc.
   - Select the option with highest likelihood
   - No generation required
2. **Method 2: Constrained Generation**
   - Prompt: "Question: ... Answer: [A/B/C/D]"
   - Constrain model to only generate A, B, C, or D tokens
   - Check if generated token matches correct answer

Evaluation Inconsistencies

Different evaluation methods can yield **vastly different** results:

Model	HELM	Harness	Original
LLaMA-65B	63.7%	48.8%	63.6%
Falcon-40B	57.1%	52.7%	55.8%
LLaMA-30B	58.3%	45.7%	58.4%
GPT-NeoX-20B	25.6%	33.3%	26.2%

The same model can show accuracy differences of up to 15 percentage points!

6.4 Major Evaluation Challenges

6.4.1 1. Prompt Sensitivity

Prompt Variability

LLM performance is highly sensitive to:

- Exact wording of questions
- Presence/absence of examples (few-shot vs. zero-shot)
- Order of examples
- Format of answer choices

6.4.2 2. Train-Test Contamination

Data Contamination

Definition: When test data appears in the training set, inflating performance.
Detection Methods:

- **Order-based detection:** Compare likelihood of test examples in original order vs. shuffled order. If original order is significantly more likely, contamination is suspected.
- **Verbatim matching:** Search for exact test examples in training data
- **N-gram overlap analysis:** Measure overlap between train and test

Importance:

- Critical for academic benchmarking
- Less important for companies (who control their training data)
- Can lead to misleading comparisons between models

6.4.3 3. Open-Ended Generation Evaluation

Challenge of Free-Form Text

For open-ended generation (not multiple choice):

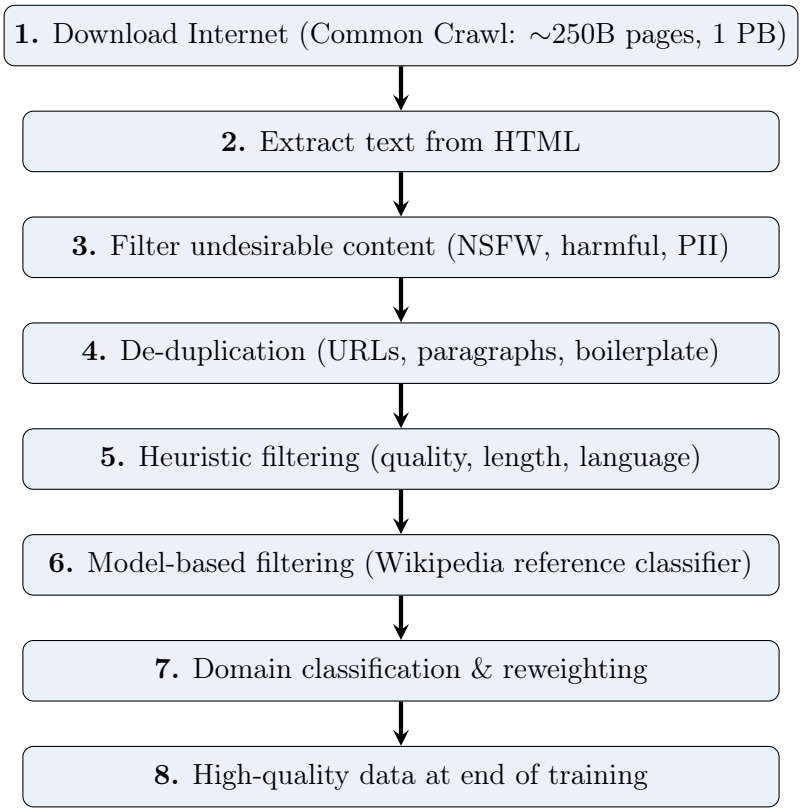
- Model may produce semantically identical but lexically different answers
- Example: "The capital is Paris" vs. "Paris is the capital"
- Requires sophisticated evaluation (human judgment or LLM-as-judge)
- We'll discuss this more in post-training section

7 Pretraining Data

Data is King

Data is arguably more important than anything else in LLM training. The quality, quantity, and composition of training data fundamentally determine model capabilities.

7.1 Data Collection Overview



7.2 Step-by-Step Data Pipeline

7.2.1 Step 1: Download the Internet

Common Crawl

Common Crawl is a widely-used web crawler that archives the internet:

- ∼250 billion web pages
- ∼1 petabyte (1,000,000 GB) of data
- Updated monthly with new pages
- Freely accessible for research

Raw Internet Reality

Raw internet data is **extremely messy**:

- Not representative of desired model behavior
- Contains spam, ads, boilerplate, malformed text
- Far from "clean" Wikipedia-quality text

7.2.2 Example of Raw Web Data

Raw HTML Example

```
<div class="header"><span>Test King World is your ultimate source for the system
x high performance server...</span>
```

This is a typical random page from Common Crawl:

- HTML tags mixed with content
- Incomplete sentences (ends with "...")
- Low-quality writing
- Boilerplate text

7.2.3 Step 2: Extract Text from HTML

Challenges:

- Removing HTML tags while preserving structure
- **Math extraction:** Very complicated but important for LLM capabilities
- **Boilerplate removal:** Headers, footers, navigation menus, ads
- Preserving formatting (lists, code blocks, tables)

7.2.4 Step 3: Filter Undesirable Content

Content Filtering

Remove:

- **NSFW content:** Not Safe For Work material
- **Harmful content:** Hate speech, violence, illegal content
- **PII:** Personally Identifiable Information (emails, phone numbers, addresses)

Methods:

- Blacklist of known problematic domains
- Trained classifiers for content categories
- Rule-based filters for PII patterns

7.2.5 Step 4: De-duplication

Why De-duplication Matters

The internet contains massive duplication:

- Same content at different URLs
- Forum headers/footers repeated millions of times
- Popular paragraphs/articles copied extensively
- Books or papers appearing on many sites

Training on duplicates:

- Wastes computational resources
- Causes models to memorize rather than generalize
- Biases distribution toward duplicated content

Challenge: Must be done at scale (billions of documents) efficiently.

7.2.6 Step 5: Heuristic Filtering

Rule-Based Quality Filters

Common heuristics include:

1. **Token distribution:** Flag documents with unusual token frequency
2. **Word length:** Filter documents with abnormally long/short average word length
3. **Document length:** Remove very short ($<$ 50 words) or suspiciously long documents
4. **Special character ratio:** High ratio suggests low quality
5. **Language detection:** Keep only desired languages
6. **Curse word density:** Filter high-profanity content
7. **Repeated phrases:** Detect spam/bot-generated content

7.2.7 Step 6: Model-Based Quality Filtering

Wikipedia Reference Classifier

A clever technique for quality assessment:

1. Extract all URLs referenced in Wikipedia articles
2. Label these as "high-quality" (positive examples)
3. Label random web pages as "low-quality" (negative examples)
4. Train a binary classifier: Wikipedia-referenced vs. random web
5. Use this classifier to score all documents
6. Keep documents with high predicted quality scores

Intuition: If it's good enough to be referenced by Wikipedia, it's probably high quality!

7.2.8 Step 7: Domain Classification and Reweighting

Domain Balancing

Classify documents into domains:

- Books
- Code (GitHub, programming sites)
- Science (papers, technical content)
- News
- Forums/Social Media
- Entertainment

Then **reweight** domains based on desired model capabilities:

- **Upweight:** Code (helps reasoning!), Books, Science
- **Downweight:** Entertainment, Low-quality forums

Code and Reasoning

Key finding: Training on more code improves general reasoning abilities, not just coding skills. This is why code is heavily upweighted in modern LLM training.

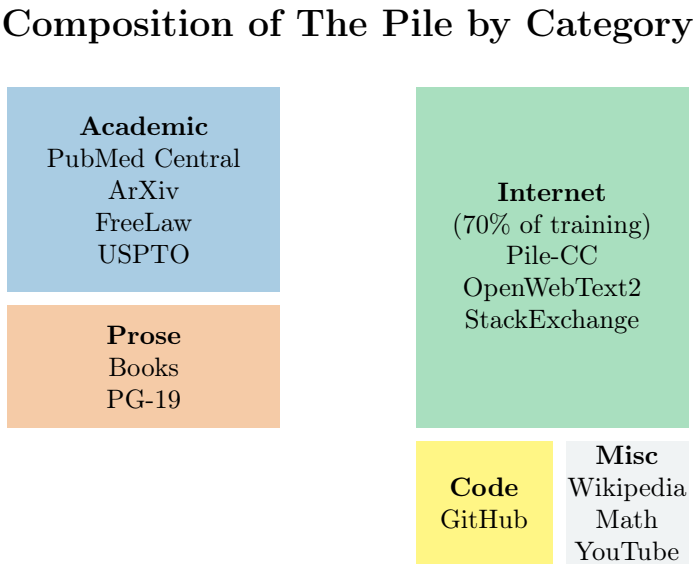
7.2.9 Step 8: High-Quality Data at End of Training

End-of-Training Strategy

At the end of pretraining:

- Decrease learning rate
- Train on **very high-quality data only**:
  - Wikipedia (overfit intentionally)
  - Human-curated content
  - High-quality books
- This "fine-tunes" the model on the best available data
- Also useful for extending context length

7.3 Data Composition: The Pile



Note: The Pile is 280B tokens (∼800 GB)

7.4 Data Scale: Historical and Current

Dataset/Model	Tokens	Approx. Size
Early benchmarks (2018)	150B	800 GB
The Pile (Academic)	280B	800 GB
LLaMA 2	2T	10 TB
LLaMA 3 (2024)	15T	80 TB
GPT-4 (estimated)	∼13T	∼70 TB
Current state-of-art	**15T**	**80 TB**

Table 1: Evolution of pretraining dataset sizes

Scale Progression

Dataset sizes have grown **100-fold** in just 5-6 years:

- 2018: ∼150B tokens
- 2024: ∼15T tokens
- Filtering reduces raw data by 10-100×

7.5 Organizational Requirements

Team Size and Resources

For a major LLM project (e.g., LLaMA scale):

- **Total team size:** ∼70 people
- **Data team:** ∼15 people (20-25% of team)

- **Compute:** Massive CPU clusters for data processing
- **Storage:** Petabytes of storage infrastructure

Key insight: Data teams are often *larger* than modeling teams!

7.6 Open Research Questions

Unsolved Problems in Data

The data problem is **far from solved**. Major open questions:

1. **Efficient processing:** How to process petabytes faster and cheaper?
2. **Optimal domain balancing:** What is the best mix of domains?
3. **Synthetic data:** Can we generate high-quality training data?
   - Becoming critical as we exhaust internet data
   - How to generate diverse, accurate synthetic data?
4. **Multimodal data:** How does image/video/audio data improve text models?
5. **Data valuation:** Which data is most valuable? How to prioritize?
6. **Copyright and privacy:** Legal and ethical data sourcing

Industry Secrecy

Companies are **extremely secretive** about data:

- Competitive advantage lies in data quality and composition
- Copyright liability concerns (e.g., training on books)
- Proprietary filtering and processing techniques

This makes it difficult for academia to replicate results or advance the field.

7.7 The Data Scarcity Problem

Running Out of Data

A looming crisis in LLM training:

- Current models train on ∼15T tokens (∼80 TB)
- High-quality internet text is **finite**
- Estimates suggest we may exhaust suitable training data by 2025-2026
- This is driving intense interest in:
  - Synthetic data generation
  - Multimodal training (images, videos as additional signal)
  - More efficient use of existing data

– Data augmentation techniques

8 Systems Considerations

Systems are Critical

With models containing billions of parameters and trained on trillions of tokens, **systems engineering** is now as important as algorithm design.

8.1 Why Systems Matter

1. **Model Scale:**
   - Models: 7B to 175B+ parameters
   - Single GPU memory: 16-80 GB
   - Models often don't fit on single GPU
2. **Training Time:**
   - Training can take weeks to months
   - Distributed across hundreds/thousands of GPUs
   - Failures must be handled gracefully
3. **Cost:**
   - Training cost: $1M to $100M+
   - Inference cost: Serving millions of requests
   - Efficiency directly impacts viability

8.2 Key Systems Challenges

1. Distributed Training: Model/data/pipeline parallelism

2. Memory Optimization: Gradient checkpointing, mixed precision

3. I/O Efficiency: Fast data loading from disk/network

4. Fault Tolerance: Checkpointing, recovery from failures

5. Inference Optimization: Quantization, KV-cache, batching

Systems as a Differentiator

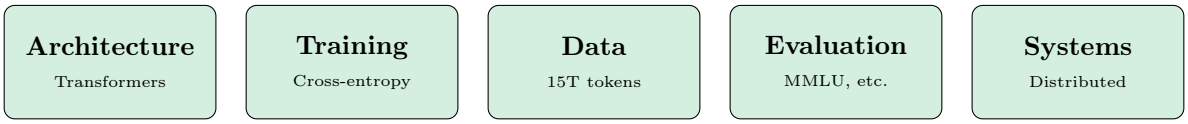
Many of the differences between leading LLM providers come down to systems engineering:

- Faster training = more experiments = better models
- Lower inference cost = more accessible products
- Better reliability = better user experience

9 Summary and Key Takeaways

9.1 The Five Pillars of LLM Training

Building Successful LLMs Requires All Five

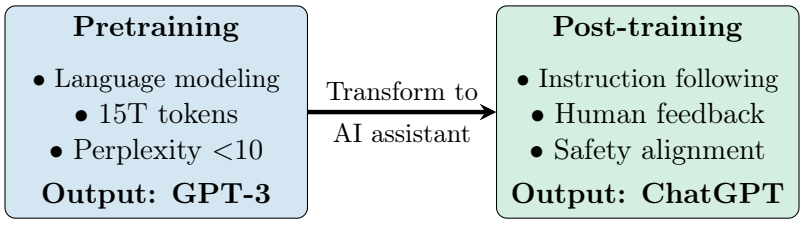


9.2 Critical Insights

What Really Matters

1. **Data is king:** Quality and composition of training data is paramount
2. **Scale matters:** Modern LLMs train on 15T+ tokens with 10B+ parameters
3. **Evaluation is hard:** Prompt sensitivity, contamination, and methodology matter enormously
4. **Systems enable scale:** Without efficient systems, modern LLMs would be impossible
5. **Industry-academia gap:** Much critical knowledge (especially about data) remains proprietary

9.3 From Pretraining to Post-training



Looking Ahead

This guide covered **pretraining**—the foundation of LLMs. The next critical phase is **post-training**, which transforms a base language model into an AI assistant like ChatGPT. Post-training involves:

- Instruction fine-tuning

- Reinforcement Learning from Human Feedback (RLHF)
- Safety and alignment procedures
- Specific capability improvements

9.4 Practical Recommendations

For Practitioners

If you're building or fine-tuning LLMs:

1. **Invest in data infrastructure:** Build robust pipelines for filtering, deduplication, and quality assessment
2. **Establish clear evaluation:** Define metrics and benchmarks before training begins
3. **Don't neglect tokenization:** Poor tokenization can cripple model performance, especially on math and code
4. **Monitor perplexity during development:** Even if not used for final benchmarking, it's invaluable for debugging
5. **Start with academic datasets:** The Pile and similar datasets are good starting points before building custom data pipelines
6. **Prioritize systems early:** Efficient training systems pay dividends throughout the project

A Mathematical Reference

A.1 Key Formulas

A.1.1 Autoregressive Decomposition

$$p(x_1, \ldots, x_L) = \prod_{i=1}^{L} p(x_i | x_{1:i-1})$$

A.1.2 Cross-Entropy Loss

$$\mathcal{L}(x_{1:L}) = -\sum_{i=1}^{L} \log p(x_i | x_{1:i-1})$$

A.1.3 Log-Likelihood

$$\log p(x_{1:L}) = \sum_{i=1}^{L} \log p(x_i | x_{1:i-1})$$

A.1.4 Perplexity

$$\text{PPL}(x_{1:L}) = 2^{\frac{1}{L}\mathcal{L}(x_{1:L})} = \exp\left(\frac{1}{L}\sum_{i=1}^{L} -\log p(x_i | x_{1:i-1})\right)$$

$$\text{PPL}(x_{1:L}) = \left[\prod_{i=1}^{L} p(x_i|x_{1:i-1})\right]^{-1/L}$$

A.2 Notation Guide

Symbol	Meaning
x_i	The i-th token in a sequence
$x_{1:L}$	Sequence of tokens from position 1 to L
$x_{1:i-1}$	Context: all tokens before position i
L	Length of the sequence
$\|V\|$	Vocabulary size (number of distinct tokens)
d	Dimension of embedding/hidden vectors
$p(\cdot)$	Probability distribution
$\mathcal{L}$	Loss function
θ	Model parameters

B Additional Resources

B.1 Recommended Reading

- **Transformers:** "Attention Is All You Need" (Vaswani et al., 2017)
- **GPT Series:** OpenAI's GPT-2 and GPT-3 papers
- **LLaMA:** Meta's LLaMA and LLaMA 2 technical reports
- **Evaluation:** "Holistic Evaluation of Language Models" (HELM paper)
- **Data:** "The Pile: An 800GB Dataset of Diverse Text for Language Modeling"

B.2 Online Courses and Tutorials

- Stanford CS224N: Natural Language Processing with Deep Learning
- Stanford CS336: Language Modeling from Scratch
- Stanford CS324: Large Language Models
- Lena Voita's NLP Course: `https://lena-voita.github.io/nlp_course.html`

B.3 Practical Tools

- **HuggingFace Transformers:** Pretrained models and tokenizers
- **PyTorch / JAX:** Deep learning frameworks
- **Datasets:** The Pile, C4, RedPajama
- **Evaluation:** HELM, EleutherAI LM Evaluation Harness

C Glossary

Autoregressive Model
A model that generates sequences one token at a time, conditioning each token on all previous tokens.

BPE (Byte Pair Encoding)
A tokenization algorithm that iteratively merges the most frequent pairs of tokens.

Context Window
The maximum number of tokens a model can process at once (e.g., 2K, 4K, 128K tokens).

Cross-Entropy Loss
The standard loss function for classification tasks, measuring the difference between predicted and target distributions.

Fine-tuning
Additional training of a pretrained model on a specific task or dataset.

LLM (Large Language Model)
A neural network-based language model with billions of parameters trained on vast amounts of text.

MMLU
Massive Multitask Language Understanding—a comprehensive benchmark with 57 subjects.

Perplexity
An evaluation metric representing the number of tokens the model is "hesitating between" when making predictions.

Post-training
The phase after pretraining where models are aligned with human preferences and made into assistants.

Pretraining
The initial training phase where models learn from large unlabeled text corpora.

Token
The basic unit of text processed by language models (typically 3-4 characters).

Transformer
The neural network architecture underlying all modern LLMs, based on self-attention mechanisms.